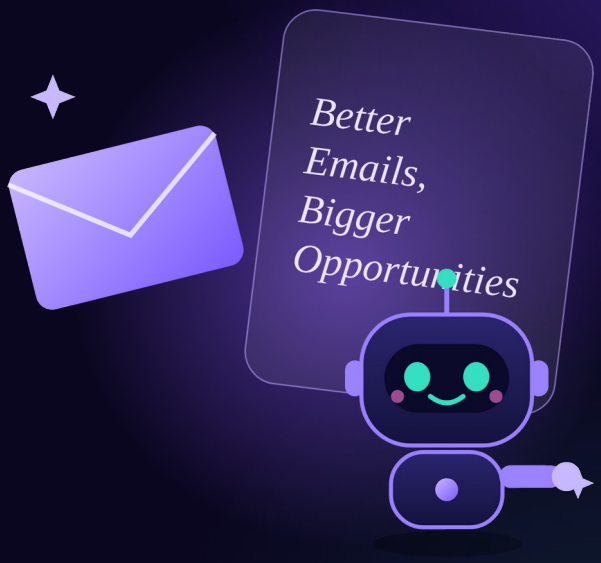
AI EMAIL GENERATOR

LetterLy

Project Report

Turn your thoughts into perfect emails



Prepared by

Anushay Ayat

SMIT (Saylani Mass IT Training)

Artificial Intelligence and Data Science

October 2026

Contents

Abstract 3
1. Introduction 4
1.1 Background and motivation 4
1.2 Problem statement 4
1.3 Objectives 4
1.4 Scope of the project 4
2. Requirements 5
2.1 Assignment requirements and how they are met 5
2.2 Functional requirements 5
2.3 Non-functional requirements 6
3. System Design 7
3.1 Architecture 7
3.2 Technology stack 7
3.3 Request flow 8
3.4 Project structure 8
3.5 Data model 8
3.6 API design 10
4. Implementation 11
4.1 The FastAPI backend 11
4.2 Prompt design and LLM integration 11
4.3 The storage layer 12
4.4 Private spaces for visitors 12
4.5 Frontend implementation 12
4.6 Feature walkthrough 13
5. User Interface and Design 19
5.1 Brand and design system 19
5.2 Landing page 19
5.3 Dashboard layout and themes 20
5.4 Responsive design 20
5.5 Accessibility 21
6. Testing 22
6.1 Strategy 22
6.2 Automated browser tests 22
6.3 Automated backend tests 22
6.4 Manual checklist for the live deployment 23
7. Deployment 24
7.1 How the deployment works 24
7.2 Environment variables 24
7.3 Steps 24
7.4 Security considerations 24
8. Challenges and Solutions 25
9. Limitations and Future Work 26
9.1 Limitations 26
9.2 Future work 26
10. Conclusion 26
References 26
Appendix A. Setup Guide 27
Appendix B. Quick User Guide 27

Abstract

LetterLy is a web application that turns a few details, namely who the email is for, what it should achieve and which tone to use, into a complete and ready-to-send email. The writing is done by a large language model through the Groq API, while a FastAPI backend validates the request, builds the prompt, checks the answer and stores the result.

The assignment asked for a simple FastAPI application with three inputs (recipient name, email purpose and tone) and an HTML page that displays the generated email. LetterLy meets this requirement and extends it into a complete writing assistant: it can reply to a pasted email, summarise a long thread, rewrite a draft on request, suggest subject lines, score the quality of an email, detect its tone and fix its grammar. It also provides ten ready-made templates, favourites and tags, a searchable history, export to Word, PDF, text and the mail app, voice input, an analytics page and a settings page. Everything is available in English and Urdu.

The interface follows a purple and pink design system with a dark landing page, a light dashboard, a dark theme and a layout that adapts to phones. The project keeps its data in SQLite during development and in Firebase Firestore when deployed on Vercel, and it is packaged with a README, a setup guide and a deployment guide. This report describes the problem, the requirements, the design, the implementation, the testing approach, the deployment process, the challenges met along the way and the possible future work.

Project snapshot

Table 1. *Key facts about the project*

Item	Details
Project	LetterLy, an AI email generator
Author	Anushay Ayat, SMIT (Saylani Mass IT Training), Artificial Intelligence and Data Science
Backend	Python with FastAPI, 22 API endpoints
AI model	Groq API, llama-3.3-70b-versatile, JSON mode
Storage	SQLite for local work, Firebase Firestore in production
Frontend	Plain HTML, CSS and JavaScript, no build step
Screens	A landing page and a dashboard with six views
Languages	English and Urdu, including right-to-left layout
Size	About 3,300 lines of code in 16 project files
Hosting	Prepared for Vercel with Firebase Firestore

1. Introduction

1.1 Background and motivation

Email is still the main way in which students, job seekers and professionals communicate formally. A good email is clear, polite and suited to its reader, but writing one takes time and care. The right wording for a teacher is different from the wording for a colleague, and writing in a second language makes the task harder. Many people also write the same kinds of email again and again: a leave request, a job application, a follow-up after an interview, a request for an extension.

Large language models can draft such emails in seconds. A general chat window, however, leaves most of the work to the user: the context must be explained every time, the tone must be tuned by trial and error, and the result has to be copied by hand into a mail program. LetterLy wraps the language model in a focused tool that asks only for what matters and then helps with everything that comes after the first draft.

1.2 Problem statement

The problem addressed by this project can be stated as follows: how can a user produce a clear, correctly toned and correctly formatted email quickly, in English or Urdu, and then improve, reuse and export it without leaving the application? This breaks down into five smaller problems:

- **Drafting.** Turn a short description of the purpose into a complete email with a greeting, a body and a sign-off.
- **Tone.** Match the wording to the reader, from friendly to strictly formal.
- **Repetition.** Reuse common emails through templates instead of starting from nothing.
- **Replying.** Understand a received email and answer all of its points.
- **Quality.** Check and polish a draft before sending it.

1.3 Objectives

- Build a FastAPI backend that accepts a recipient name, an email purpose and a tone, and returns an email written by an LLM.
- Display the generated email on a clear HTML page where it can be edited.
- Add tools that improve a draft: refine, translate, summarise, score, detect tone and fix grammar.
- Let users organise their work with templates, favourites, tags, history and analytics.
- Design a polished, responsive interface with a landing page and a dashboard.
- Make the project easy to run locally and ready to deploy on Vercel with Firebase.
- Test the code and document it so that others can set it up and understand it.

1.4 Scope of the project

The project covers the generation, editing, organisation and export of emails. It deliberately does not send emails by itself: the "Mail" button opens the user's own mail program with the subject and body filled in, which keeps the user in control of what is sent. User accounts and passwords are also outside the scope. Instead, each browser receives a private space through a cookie, which is enough for a personal tool and keeps the project simple. Native mobile apps are not part of the project, but the interface is fully responsive.

2. Requirements

2.1 Assignment requirements and how they are met

The table below lists each requirement of the assignment together with the place in LetterLy where it is satisfied.

Table 2. *Assignment requirements and their implementation*

Requirement	How LetterLy meets it	Where
FastAPI backend	A FastAPI application with 22 endpoints, input validation and error handling	main.py
Input: recipient name	Required field in the form and in the API request	POST /api/generate
Input: email purpose	Multi-line field of up to 800 characters, with templates and voice input	POST /api/generate
Input: tone	Professional, Friendly or Formal, chosen with a segmented control	POST /api/generate
Generate a complete email with an LLM	Groq with llama-3.3-70b-versatile, answering in JSON with a subject and a body	ask_email()
Display it on a simple HTML page	The "Generated email" card of the dashboard, with an editable subject and body	public/app.html
GitHub repository link	A repository with README, setup and deployment guides	Submission
Screenshot of the working app	Screenshots of every screen are included in this report	Chapters 4 and 5

2.2 Functional requirements

Table 3. *Functional requirements*

ID	Requirement
FR1	Generate an email from a recipient name, a purpose and a tone.
FR2	Offer optional length (short, medium, long), language (English or Urdu) and sender name.
FR3	Show the subject and body, let the user edit both and save the edits automatically.
FR4	Write a reply to a pasted email, optionally guided by what the reply should say.
FR5	Refine a draft: shorten, lengthen, translate between English and Urdu, or apply a custom instruction.
FR6	Suggest three alternative subject lines.
FR7	Give a quality score from 0 to 100 with a verdict and improvement tips.
FR8	Describe the tone of an email as percentages.
FR9	Fix spelling and grammar without changing the meaning.
FR10	Summarise an email thread into a gist, questions to answer and deadlines.
FR11	Provide built-in templates and let the user save and delete their own.
FR12	Keep a history with search and filters.
FR13	Support favourites and up to five tags per email.
FR14	Export or share an email: copy, text file, Word file, PDF and mail app.

ID	Requirement
FR15	Accept the email purpose by voice where the browser allows it.
FR16	Show analytics: emails written, average quality, time saved, tone mix and weekly activity.
FR17	Store settings: name for sign-offs, default tone and language, typing animation and score display.

2.3 Non-functional requirements

- **Usability.** The dashboard must be usable with a keyboard, must respect the reduced-motion setting of the system and must work on phone screens without sideways scrolling.
- **Performance.** The interface must stay responsive while the AI works. The waiting time itself depends on the Groq service.
- **Security and privacy.** API keys must never appear in the code or in GitHub. Visitors of a public deployment must not see one another's emails.
- **Portability.** The project must run locally with no database setup and must also run on Vercel, where the disk is read-only.
- **Reliability.** Every request must be validated, and failures of the AI service must produce clear messages rather than crashes.
- **Maintainability.** Storage, API logic and interface code must be separated, and the project must include documentation.

3. System Design

3.1 Architecture

LetterLy has three layers. The browser holds the landing page and the dashboard. A FastAPI application exposes a JSON API, validates every request, builds prompts and talks to the AI service. A storage layer keeps the emails, templates and settings. The storage layer hides the difference between SQLite, which is used on a developer's computer, and Firebase Firestore, which is used when the application runs on Vercel.

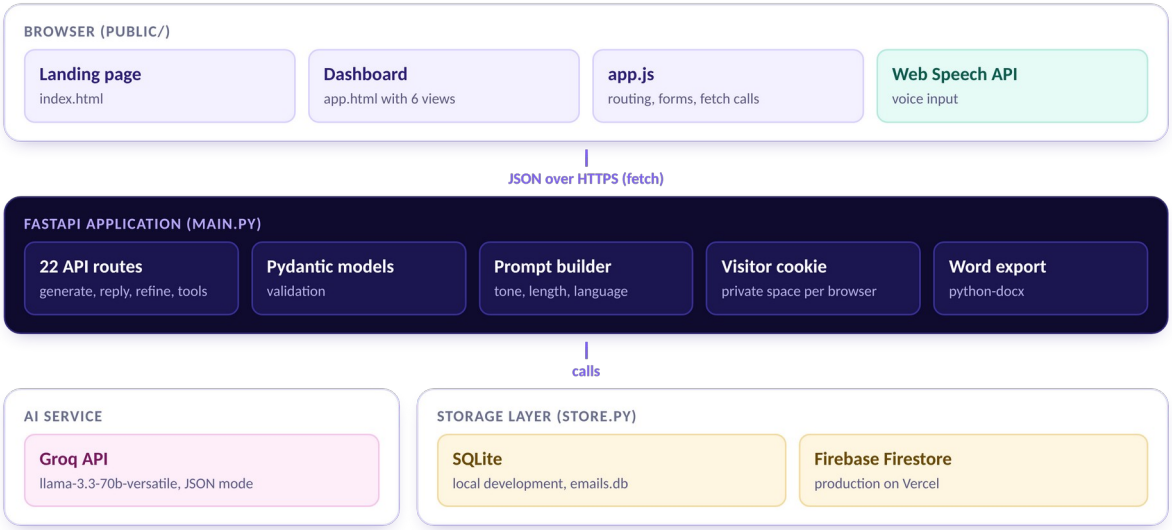


Figure 1. *Architecture of LetterLy: browser, FastAPI application, Groq API and storage layer*

3.2 Technology stack

Table 4. *Technology stack*

Layer	Technology	Why it was chosen
Backend	Python, FastAPI, Pydantic	Fast to write, automatic request validation and interactive API documentation at /docs
AI	Groq API, llama-3.3-70b-versatile	Fast inference, a free tier for students and a JSON response mode
Local storage	SQLite	Part of Python, needs no setup and is ideal for development
Production storage	Firebase Firestore	A cloud database that works with Vercel's read-only filesystem
Word export	python-docx	Creates real Word files, including right-to-left paragraphs for Urdu
Frontend	HTML, CSS and JavaScript	No build step, fast loading and easy to understand
Voice input	Web Speech API	Built into Chromium-based browsers
Hosting	Vercel	Serves pages from a CDN and runs the API as a function

3.3 Request flow

Generating an email takes six steps, shown in Figure 2. The browser first checks that the required fields are filled in and sends a POST request. FastAPI validates the data with Pydantic models, writes a prompt that includes tone and length guidance and asks Groq for a JSON answer. The server parses and checks that answer, stores the email for the current visitor and returns it. The page then shows the email with a short typing animation.

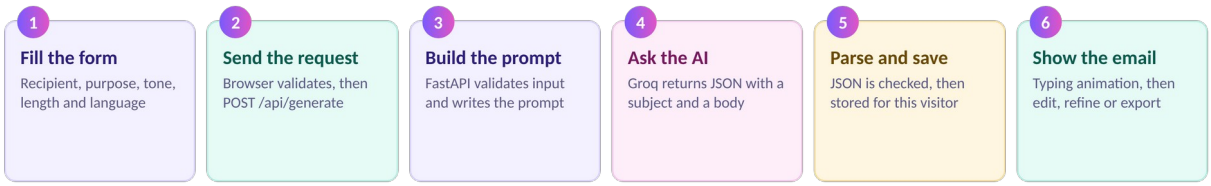


Figure 2. *The six steps from the form to the displayed email*

3.4 Project structure

The project is organised so that each part has one job. The Python files are in the project root, and everything that the browser loads is inside the public folder, which Vercel serves from its content delivery network.

```
ai-email-generator/
  main.py                    FastAPI app, prompts, all API routes
  store.py                   SQLite and Firestore storage
  requirements.txt           packages used on Vercel
  requirements-dev.txt       adds uvicorn for local work
  vercel.json                Vercel settings (clean URLs)
  .env.example               template for local secrets
  tools/
    encode_firebase_key.py   turns a Firebase key into one line
  public/
    index.html               landing page
    app.html                 dashboard
    static/css/              base.css, landing.css, app.css
    static/js/               landing.js, app.js
```

3.5 Data model

An email is the central record. The same fields are used in SQLite tables and in Firestore documents, which is why the API looks identical on both storage back ends.

Table 5. *Fields of an email record*

Field	Type	Meaning
`id`	number	Unique identifier, based on the creation time in Firestore
`kind`	text	"new" for a generated email, "reply" for a reply
`recipient_name`	text	Who the email is for
`purpose`	text	What the email should achieve
`source_text`	text	The received email, for replies
`tone, length, language`	text	The options chosen in the form
`subject, body`	text	The generated and edited email
`pinned`	yes or no	Whether the email is a favourite
`tags`	list	Up to five short labels
`quality`	number	The last quality score, empty after a rewrite
`created_at`	date and time	When the email was created

Templates store a name, a purpose, a tone, a length and a language. Settings are one small record with the sender name, the default tone and language, and two switches. In Firestore every visitor has a separate document under users, and the emails, templates and settings of that visitor live below it.

3.6 API design

The API follows common REST conventions: GET reads, POST creates or runs a tool, PUT replaces content, PATCH changes part of a record and DELETE removes. All answers are JSON except the Word export. FastAPI also publishes interactive documentation at /docs.

Table 6. *The API endpoints (22 routes in total)*

Method	Path	Purpose
GET	`/api/health`	Server status, active storage, API key check
POST	`/api/generate`	Generate an email
POST	`/api/reply`	Reply to a pasted email
POST	`/api/summarize`	Summarise an email thread
POST	`/api/emails/{id}/refine`	Shorten, lengthen, translate or custom change
POST	`/api/emails/{id}/subjects`	Three alternative subjects
POST	`/api/emails/{id}/analyze`	Quality score with tips
POST	`/api/emails/{id}/tone`	Tone breakdown
POST	`/api/emails/{id}/grammar`	Fix spelling and grammar
PUT	`/api/emails/{id}`	Save edits
PATCH	`/api/emails/{id}/meta`	Favourite and tags
GET	`/api/emails/{id}/export.docx`	Download as a Word file
GET	`/api/emails`	List, search and filter
GET	`/api/tags`	Tags in use with counts
DELETE	`/api/emails/{id}`	Delete one email
DELETE	`/api/emails`	Delete all emails
GET, PUT	`/api/settings`	Read or save settings
GET	`/api/stats`	Numbers for the analytics page
GET, POST	`/api/templates`	List or save templates
DELETE	`/api/templates/{id}`	Delete a saved template

4. Implementation

4.1 The FastAPI backend

The backend lives in main.py. It starts with Pydantic models that describe every request. The models limit the length of each field, for example a purpose must have between 3 and 800 characters and a pasted email between 10 and 6,000 characters. The choices for tone, length and language are fixed lists, so an unexpected value is rejected before it reaches the AI. Whitespace around text is removed before validation.

Endpoints are asynchronous. Calls to the AI service use the asynchronous Groq client, and calls to the storage layer run in a thread pool so that a slow database never blocks other requests. Errors of the AI service are translated into clear HTTP responses: a rate-limit error becomes status 429, a rejected key becomes status 500 with a hint to check the key, and a network problem or unreadable answer becomes status 502. The browser shows the message to the user in a notification.

4.2 Prompt design and LLM integration

A system prompt tells the model that it is an expert email writer, that it must use plain text with blank lines between paragraphs, that it must never invent facts, dates or numbers, and that it must answer only with a JSON object containing a subject and a body. The user prompt then supplies the details of the request: the language, the recipient, the purpose, a description of the tone, a description of the length and the instruction for the sign-off.

Table 7. *Tone and length guidance used in prompts*

Option	Guidance given to the model
Professional	Polished, clear and courteous; confident but never stiff
Friendly	Warm, relaxed and personable while still respectful
Formal	Highly formal and respectful, following traditional business-letter conventions
Short length	About 60 to 90 words
Medium length	About 120 to 170 words
Long length	About 200 to 280 words

Groq is asked for a JSON response. Even so, the server does not trust the answer blindly. A tolerant parser removes code fences, falls back to extracting the first JSON object from the text and then checks that both a subject and a body are present. If anything is missing, the user receives a clear message and can try again. Each task uses its own temperature, a setting that controls how varied the wording is.

Table 8. *Temperature used for each task*

Task	Temperature	Reason
Generate an email	0.7	Natural, varied wording
Refine or translate	0.5	Change the text but keep the facts
Subject suggestions	0.9	Three clearly different options
Quality score	0.3	Steady and honest scoring
Tone detection and summary	0.2	Consistent measurement and faithful summaries
Grammar fix	0.1	Correct mistakes without rewriting

Safe handling of pasted text. When a user pastes an email to reply to or a thread to summarise, the prompt states that this text is content to respond to or summarise, not instructions for the model. Instructions hidden inside a pasted email are therefore not followed.

Some tools post-process the answer. The quality tool clamps every score to the range 0 to 100. The tone tool accepts only the four known tone names and rescales the percentages so that they add up to exactly 100. Lists such as tips, questions and deadlines are cut to a maximum number of short items.

4.3 The storage layer

All reading and writing goes through store.py. It defines one set of operations (get, insert, update, delete, list, templates and settings) and two implementations. SqliteStore keeps data in a file called emails.db. It creates its tables on first use and upgrades an older database automatically by adding missing columns. FirestoreStore keeps the same data in Firebase.

The application chooses the implementation by itself. If a Firebase service account is present in the environment, Firestore is used; otherwise SQLite is used. On Vercel, where the disk is read-only, the SQLite file falls back to the temporary folder so that the application still starts if the Firebase key is missing.

Firestore does not support searching inside text, so the Firestore store loads the visitor's emails and applies the search, tag and favourite filters in Python. This is perfectly adequate for a personal tool with hundreds of emails. New records receive an identifier based on the current time in microseconds, which sorts by creation time and stays small enough to be handled exactly by JavaScript. Deleting all emails is done in batches of 400 documents.

4.4 Private spaces for visitors

A public deployment has no login, so the application gives every browser its own space. On the first request the server creates a random 32-character identifier and sends it in a cookie. The cookie is marked HttpOnly so that scripts cannot read it, SameSite=Lax to limit cross-site use and Secure when the connection uses HTTPS. On each later request the server validates the identifier with a strict pattern and uses it as the owner of all data. In Firestore the data is stored under users/{identifier}. A visitor who clears the cookies starts again with an empty space, which is an accepted limitation of this simple design.

4.5 Frontend implementation

The frontend uses no framework. The landing page is index.html. The dashboard is app.html with six views (Generate, Templates, History, Favorites, Analytics and Settings) that are shown and hidden by a small router based on the address hash, for example #/history. The behaviour of the dashboard is in app.js, which keeps its state in one object and talks to the API with a thin wrapper around fetch.

- **Safe rendering.** User and AI text is always inserted with textContent, never as HTML, which prevents script injection.
- **Autosave.** Edits to the subject and body are saved 700 milliseconds after the user stops typing.
- **Busy state.** While the AI works, buttons are disabled and the main button shows a progress label.
- **Typing animation.** A newly generated email types itself out; a click on the text shows it at once, and users can switch the effect off in Settings.
- **PDF export.** A hidden print sheet is filled with the subject and body and printed with a print-only style sheet, which keeps Urdu text correct.
- **Voice input.** The Web Speech API fills the purpose field in English or Urdu where the browser supports it.

- **Themes.** The theme is chosen from the system setting at first and then remembered in the browser.

4.6 Feature walkthrough

4.6.1 Generating an email

The dashboard opens on the Generate view. The form on the left collects the recipient, the purpose, the tone, the length, the language and an optional sender name. Template chips above the tone selector fill in a typical purpose. The generated email appears in the dark card on the right, where the subject and body are editable. Above them are the actions Favorite, Copy, Text, Word, PDF, Mail and Regenerate.

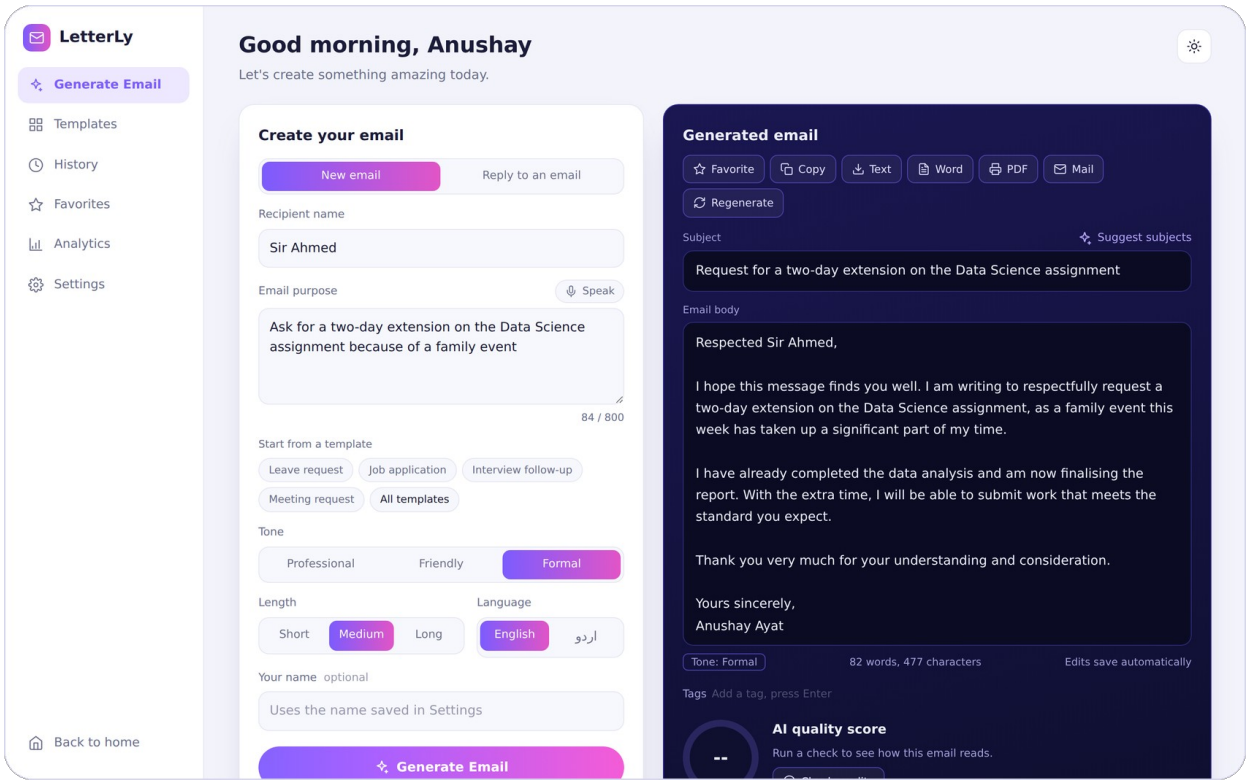


Figure 3. *The Generate view with a formal email for a deadline extension (sample data)*

4.6.2 Reply mode and thread summary

Switching to "Reply to an email" reveals a field for the received email. The user can say what the reply should cover or leave the field empty and let the AI answer every point. The original email stays available in a collapsible box above the reply. The thread summary card turns a long conversation into a gist, the questions that need an answer and any deadlines. A single button then opens the reply form with the thread and the key points already filled in.

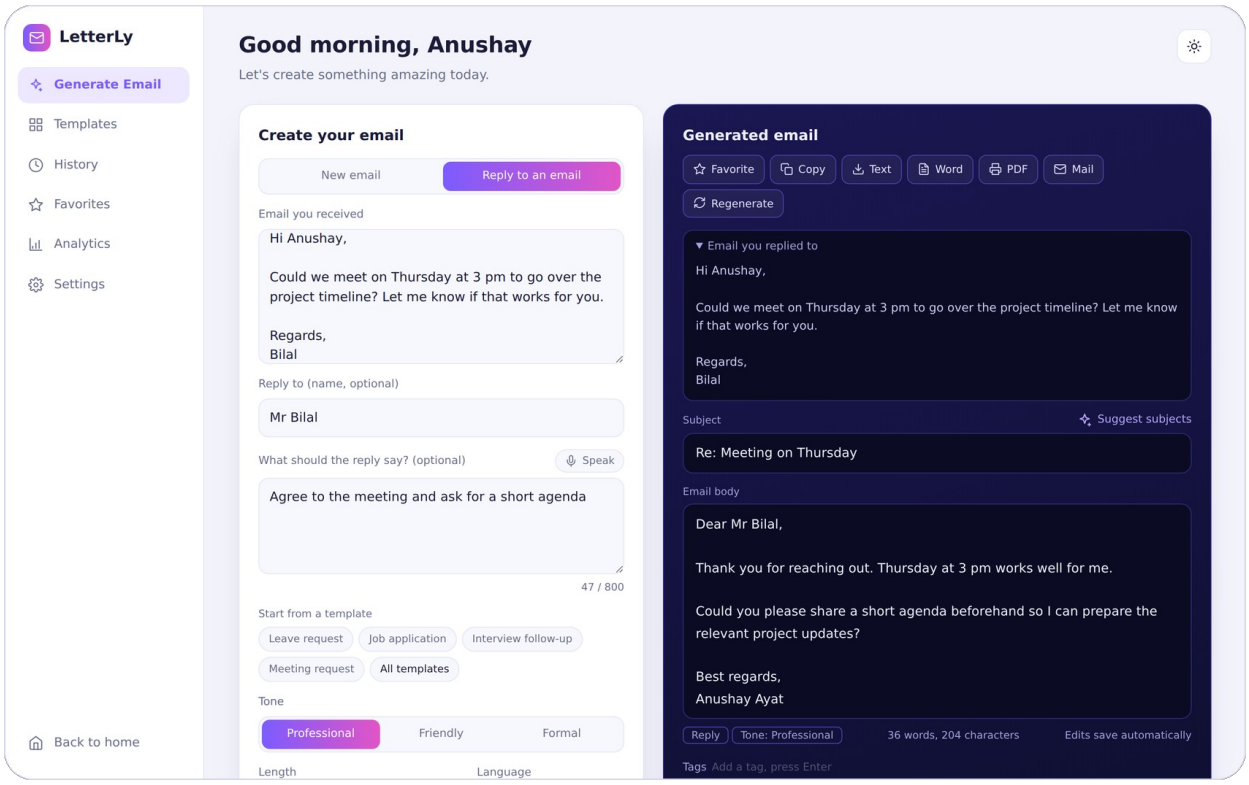


Figure 4. *Reply mode with the original email shown above the reply (sample data)*

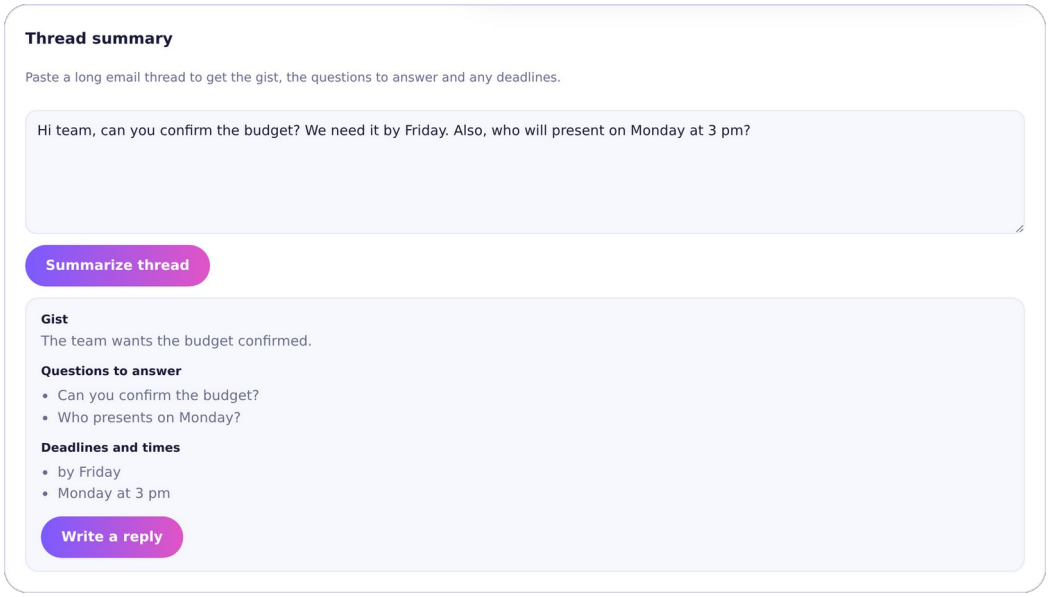


Figure 5. *The thread summary card with gist, questions and deadlines (sample data)*

4.6.3 AI tools around the draft

Below the email are eight tools. Make it shorter, Make it longer, Make it more formal, Make it more friendly and Add call to action rewrite the draft. A free text field accepts any instruction, such as "make it more polite". Translate switches between English and Urdu. Detect tone shows how professional, friendly, formal and confident the draft sounds. Fix grammar corrects spelling and punctuation and lists what it changed. The Suggest subjects link offers three alternatives, and Check quality gives a score, a verdict and tips.

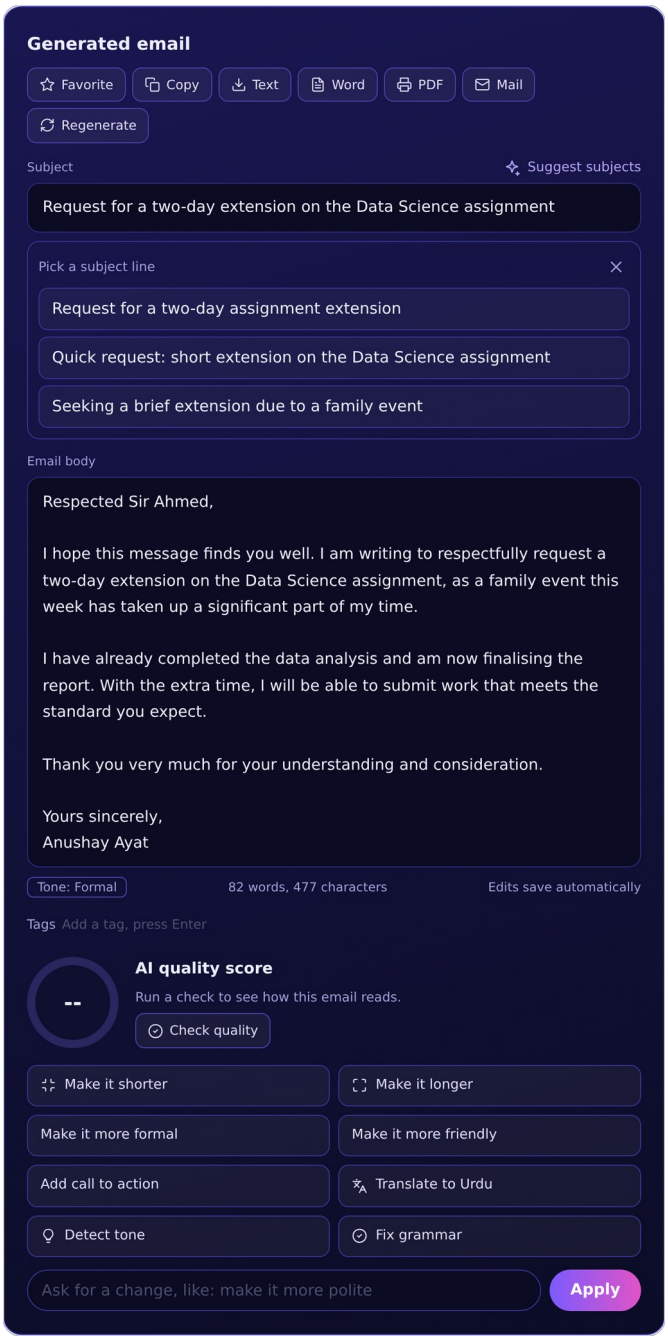


Figure 6. *Subject suggestions open under the subject line (sample data)*

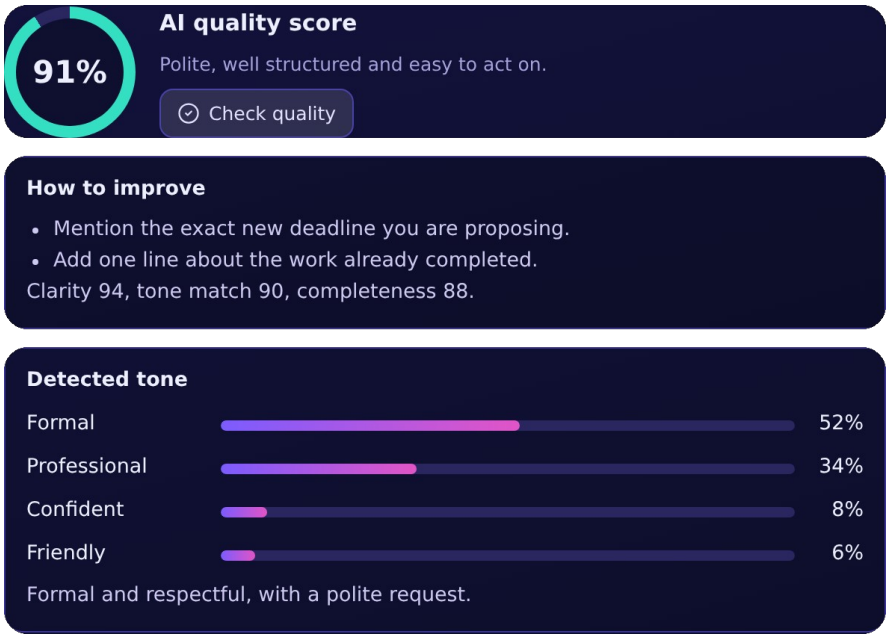


Figure 7. *Quality score with tips, and the tone detector (sample data)*

4.6.4 Templates, favourites, tags and history

Ten built-in templates cover the most common emails: leave request, job application, internship application, interview follow-up, meeting request, deadline extension, thank-you note, complaint, recommendation request and apology for delay. A user can save the current form as a template and delete it later. The History view lists every email with its category, recipient, date, tone and tags, and offers search plus filters for favourites and tags. Favourites are also listed in their own view.

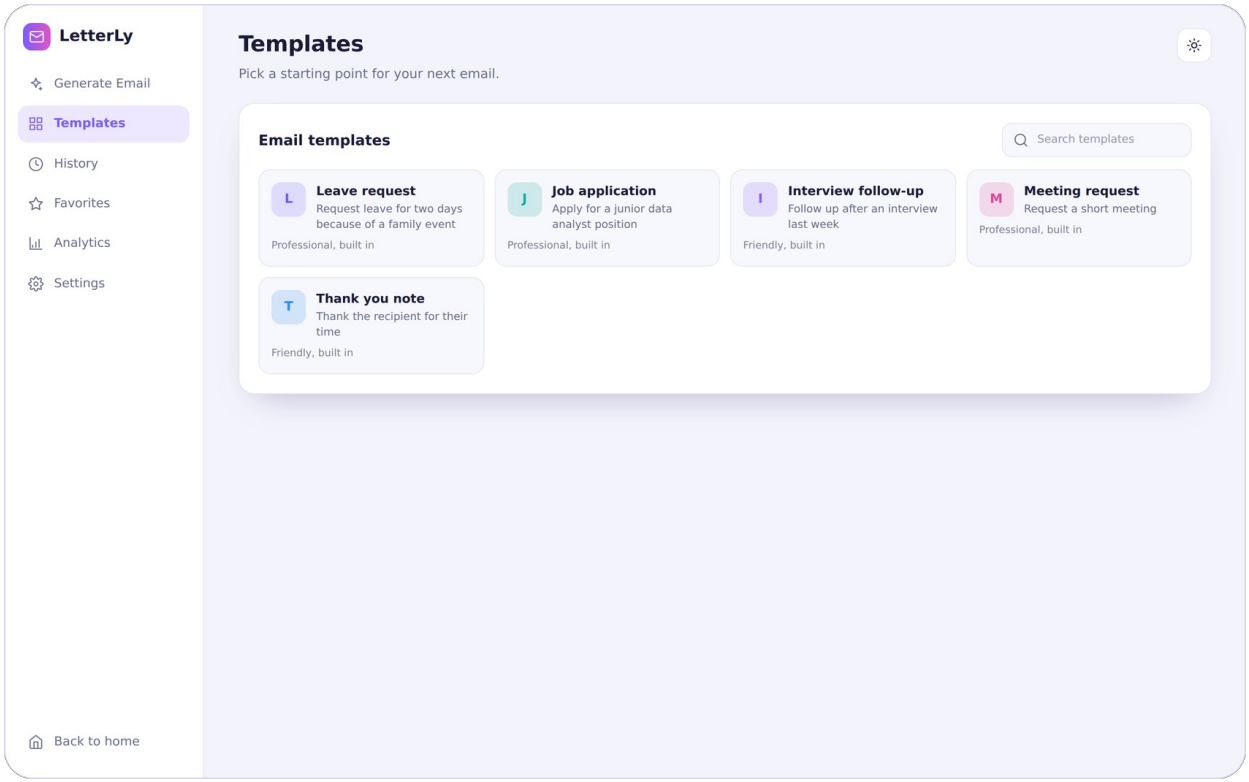


Figure 8. *The Templates view with built-in and saved templates (sample data)*

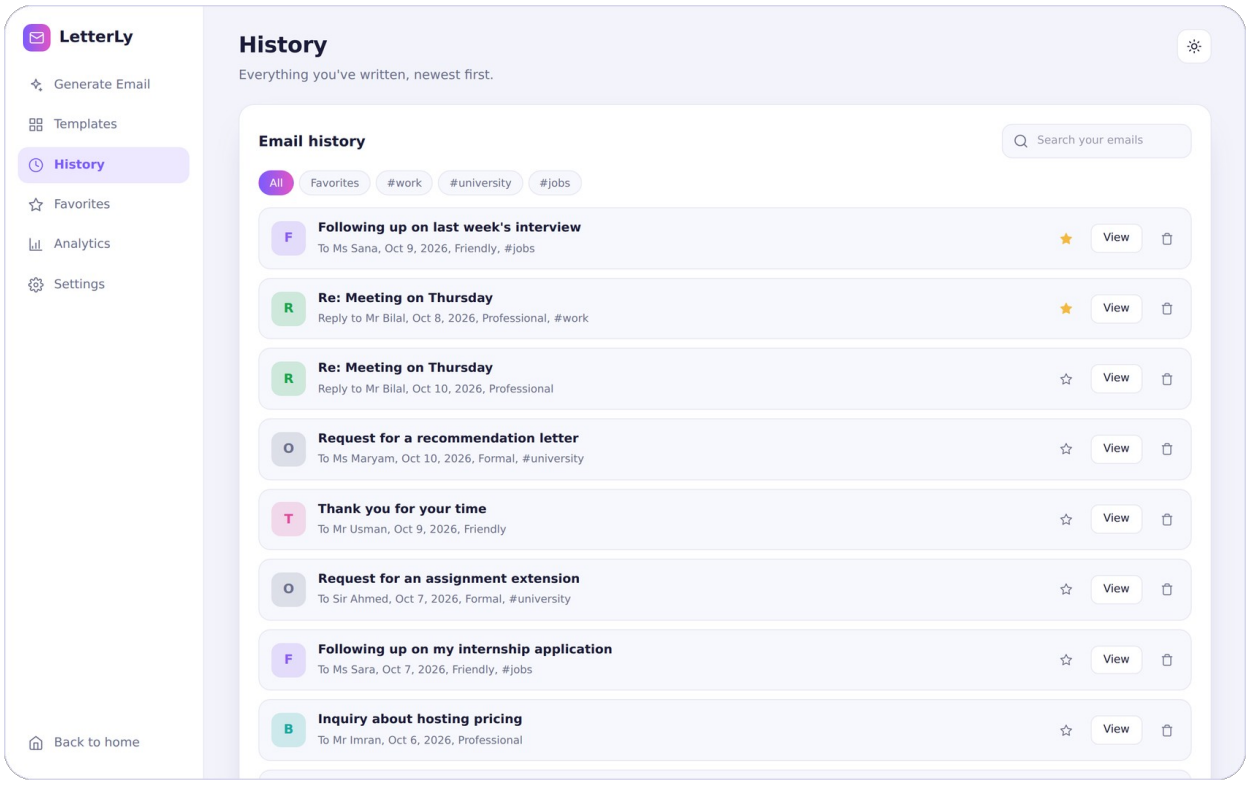


Figure 9. *The History view with search, filters, favourites and tags (sample data)*

4.6.5 Export and voice input

An email can be copied to the clipboard, downloaded as a text file, downloaded as a Word document, printed to PDF or opened in the user's mail program. The Word file is created by the server with python-docx and uses a right-to-left layout and an Urdu font for Urdu emails. For PDF the browser's print dialog is used with the option Save as PDF, because this reproduces Urdu text more reliably than building a PDF on the server. A Speak button next to the purpose field starts voice input in Chrome or Edge.

4.6.6 Analytics and settings

The Analytics view turns the history into numbers: the count of emails, the average quality score of the emails that were checked, the estimated time saved at four minutes per email, the favourite tone, the tone mix, the activity of the last seven days and the kinds of email written most often. The Settings view stores the name used in sign-offs, the default tone and language, and the switches for the typing animation and the quality score. The saved name is also used on the server when the form leaves the sender name empty.

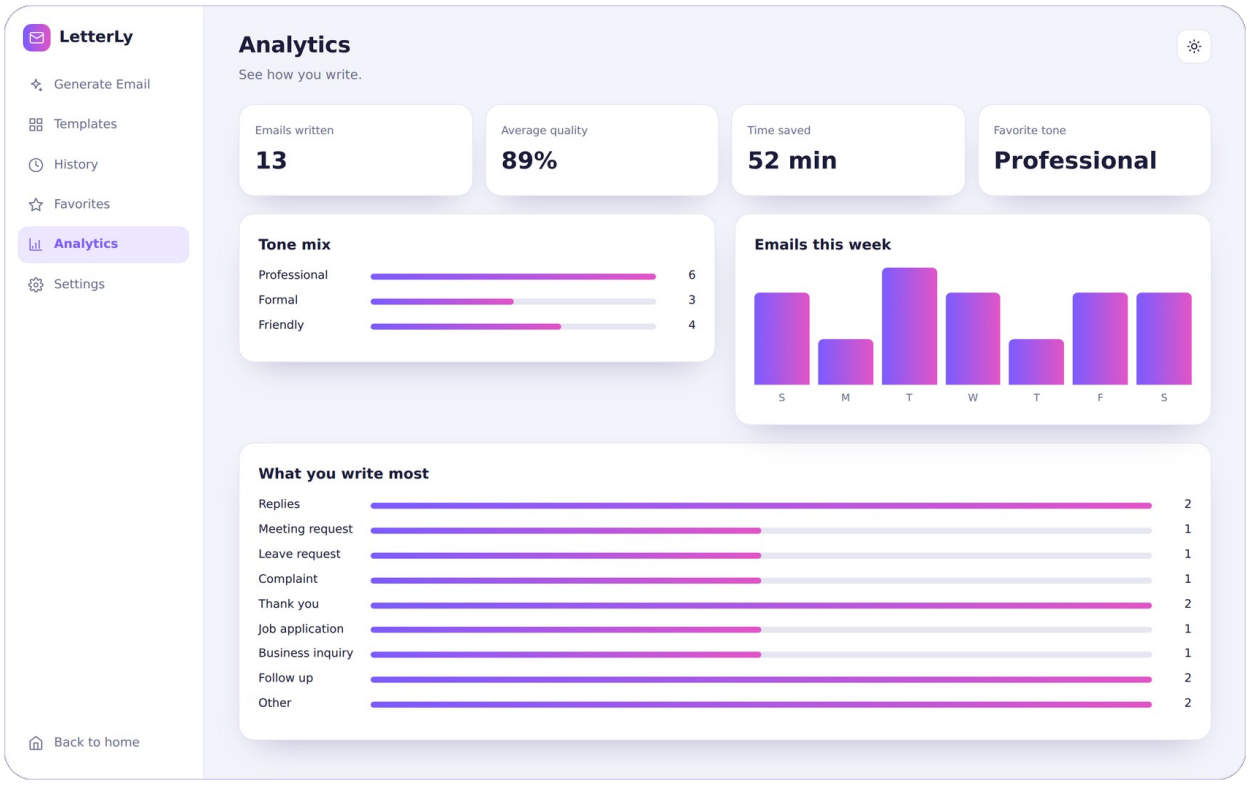


Figure 10. *The Analytics view (sample data)*

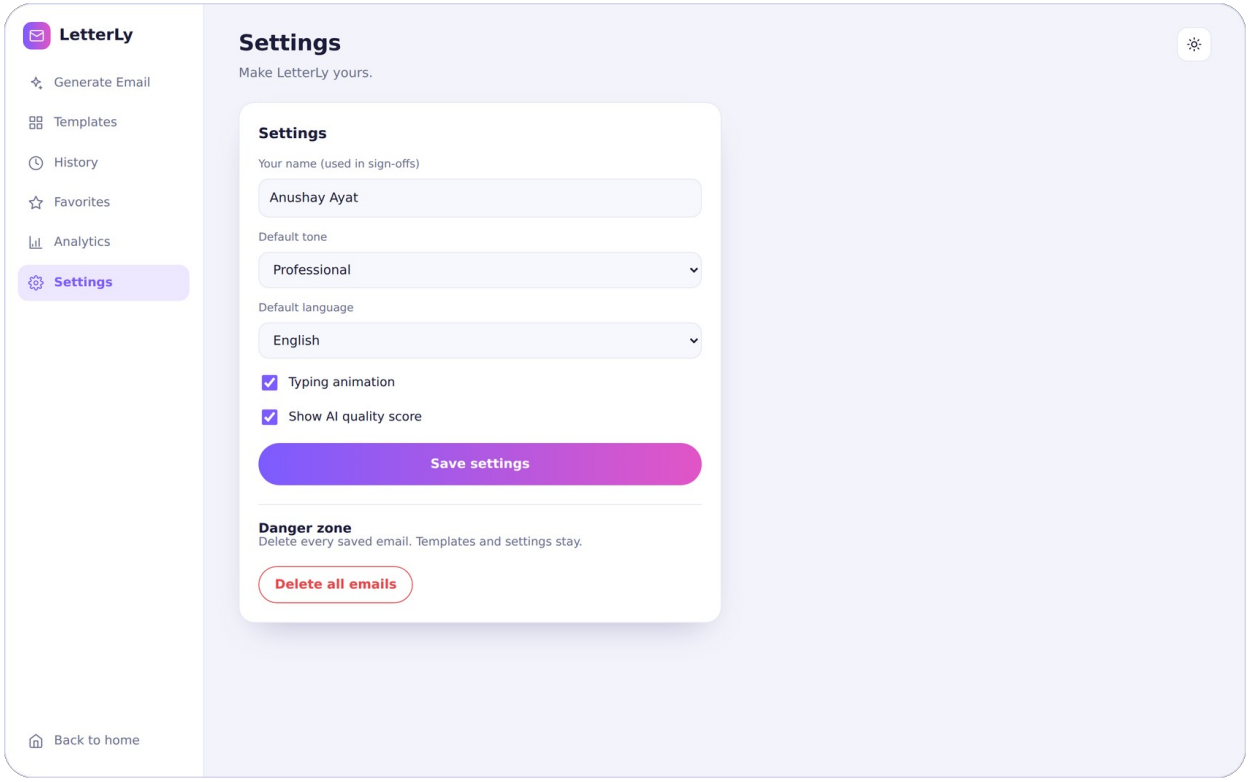


Figure 11. *The Settings view (sample data)*

5. User Interface and Design

5.1 Brand and design system

The design was prepared as a reference image before the code was written and was then implemented faithfully. It combines a dark, glowing landing page with a calm light dashboard. A violet to pink gradient is the single accent, used for buttons, active controls and charts. A friendly robot mascot gives the product a personality. Headings use the serif typeface Playfair Display and body text uses Inter.

Table 9. *Colour palette*

Colour	Value	Use
Violet	#7C5CFF	Primary accent, active navigation, links
Pink	#E255C7	Second end of the gradient, highlights
Teal	#35E0C2	Quality ring, small highlights on dark backgrounds
Ink	#1B1B3A	Text on light backgrounds
Night	#0A0620	Landing page background and the generated email card
Mist	#F3F4FB	Dashboard background

5.2 Landing page

The landing page introduces the product with the headline "Turn Your Thoughts Into Perfect Emails", a short description, three feature highlights, a prompt bar that carries the typed idea into the dashboard and quick links to popular templates. Further sections explain why to choose LetterLy, how it works in three steps and how it looks on a phone, followed by a closing quote. The robot, the envelope and the glass card float gently, the robot waves and blinks, and the hero text enters in sequence. All motion stops when the visitor prefers reduced motion.

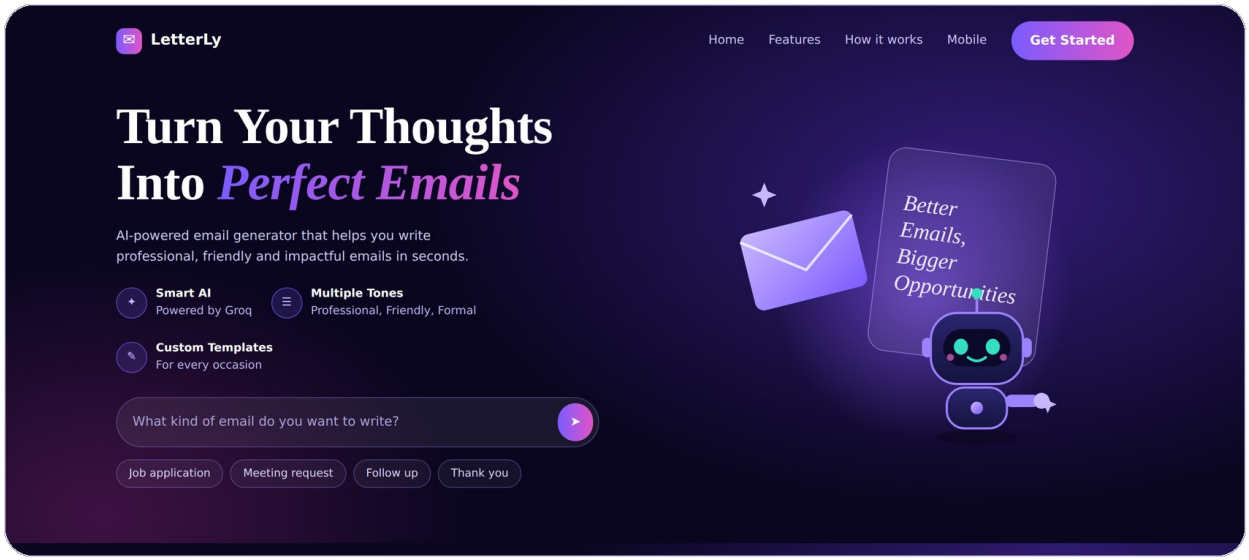


Figure 12. *The landing page hero section*

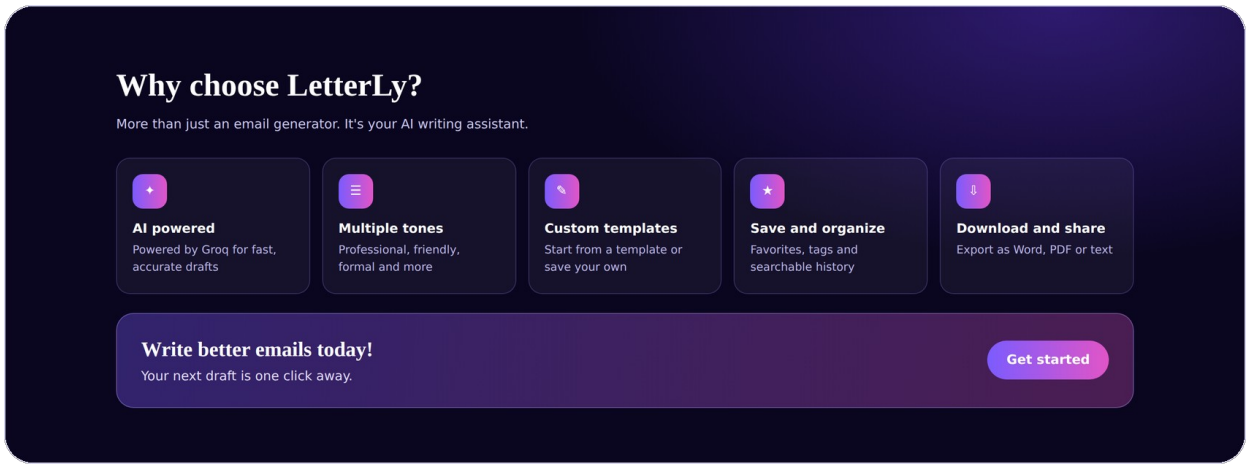


Figure 13. *The "Why choose LetterLy" section of the landing page*

5.3 Dashboard layout and themes

On wide screens the dashboard has a sidebar with the six views, a header with the greeting and a theme switch, and a main area with the form and the result side by side. The generated email card stays dark in both themes so that the email is always the focus. The dark theme changes the surrounding surfaces, text and borders.

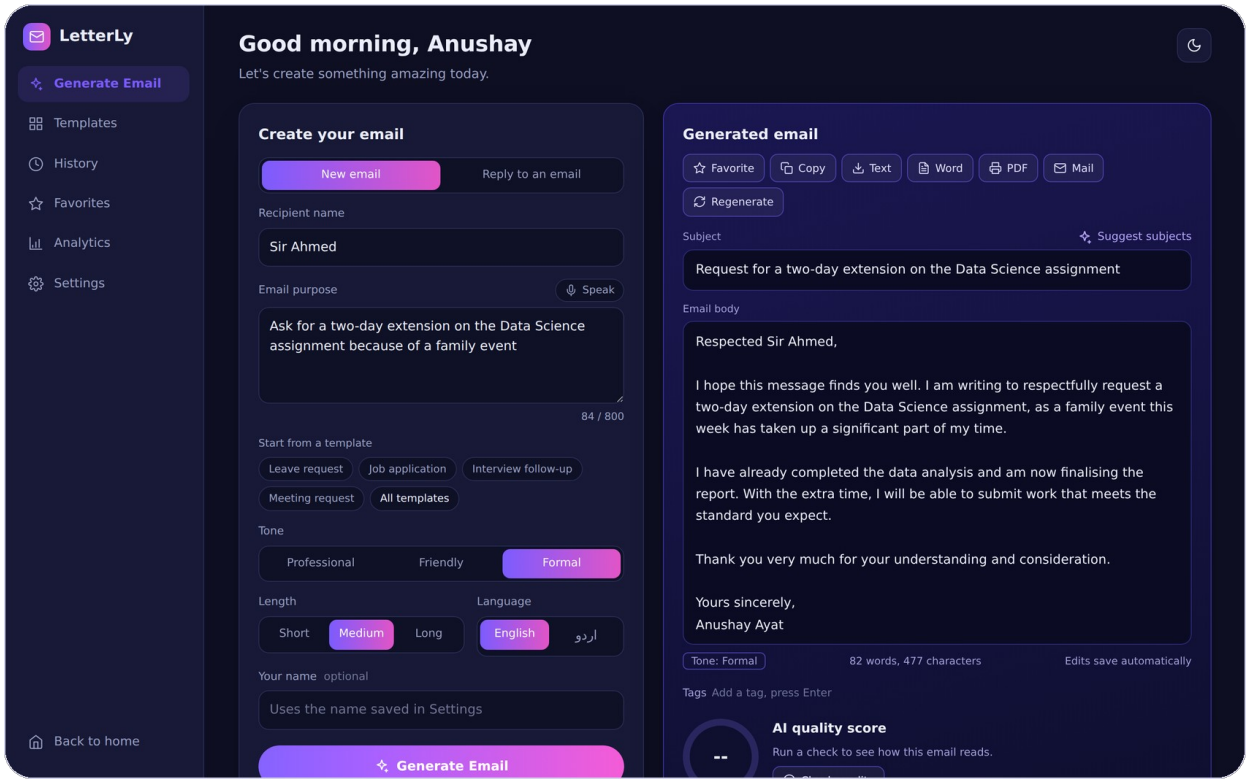


Figure 14. *The dashboard in the dark theme (sample data)*

5.4 Responsive design

Below 1100 pixels the form and the result stack vertically. Below 860 pixels the sidebar is replaced by a tab bar at the bottom of the screen with the five most-used views, as in the mobile design. Controls are large enough for touch and the layout never needs sideways scrolling.

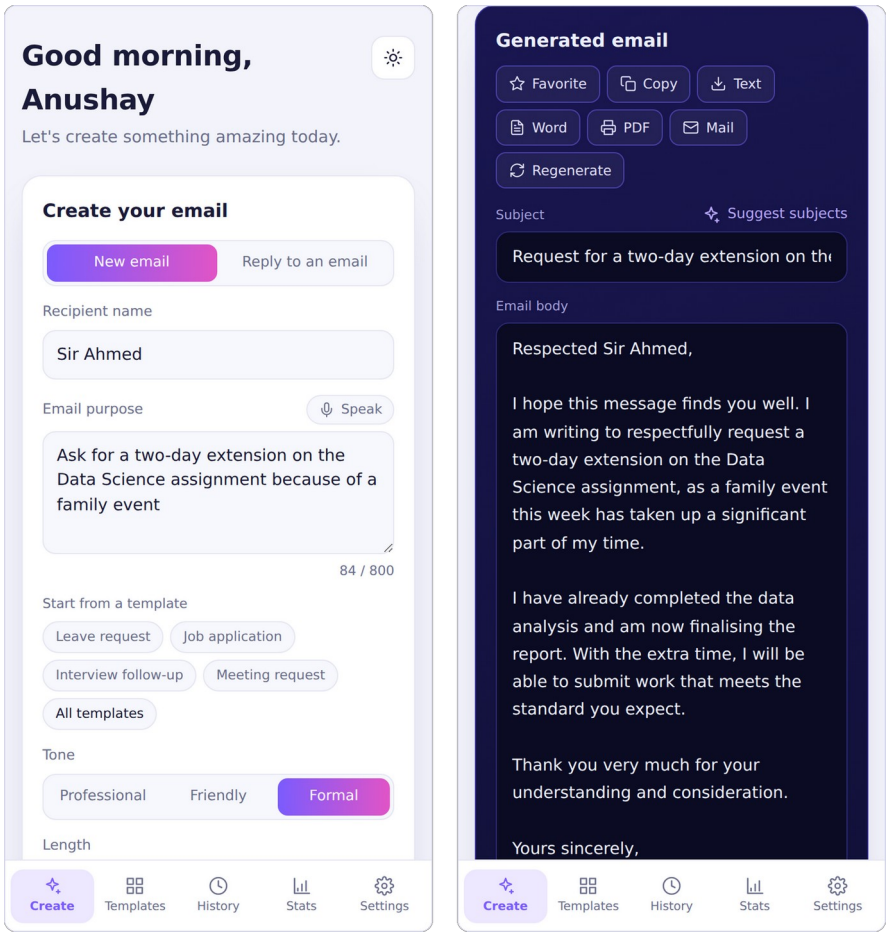


Figure 15. *The dashboard on a phone: form and generated email (sample data)*

5.5 Accessibility

- Every field has a label, and screen-reader-only labels are used where the design shows none.
- Icons that carry meaning have text alternatives, and decorative icons are hidden from assistive technology.
- Keyboard focus is shown with a clear outline on every control.
- Status messages and errors are announced through live regions.
- Animations are switched off when the system asks for reduced motion.

6. Testing

6.1 Strategy

Testing combined automated checks with a manual checklist. The automated checks run a real headless browser against the actual HTML, CSS and JavaScript, and they call the actual backend code. Two stand-ins keep the tests fast and repeatable: a mock API replaces the server in the browser tests, and a small in-memory imitation of Firestore replaces the cloud database in the backend tests. This verifies the logic of the project very thoroughly. What it cannot verify is the behaviour of the live Groq service and of the Vercel platform, which is why the manual checklist in Section 6.4 is meant to be run on a real deployment.

6.2 Automated browser tests

The browser suite contains 44 checks. It opens the landing page and the dashboard in a desktop window and in a phone-sized window and exercises every feature through the user interface.

- Landing page: headline, feature section and how-it-works steps; no sideways scroll on a phone.
- Form: validation messages, template chips, reply mode requirements and deep links from the landing page.
- Generation and refinement: generate, shorten, custom instruction, empty instruction message, Urdu and back to English.
- AI tools: subject suggestions, quality ring and tips, tone panel, grammar fix.
- Organisation: favourite, tags, saving and deleting templates, history search and filters, opening an email from history.
- Pages: favourites, analytics numbers and chart, settings and their effect on the greeting and quality display, theme switch.
- Export: the Word download starts without leaving the page and the PDF print sheet is filled.
- Phone layout: tab bar visible, sidebar hidden, navigation by tab, no horizontal overflow.

6.3 Automated backend tests

The backend suite imports the real main.py and calls its endpoint functions with a replaced AI function that returns fixed JSON. It runs twice, once with SQLite and once with the Firestore store on top of an in-memory imitation. It checks cookie handling, settings, generation, replies, editing, tags and favourites, filters, every AI tool, statistics, Word export, templates, deleting and clearing data, and, for Firestore, the isolation of one visitor from another and the deletion of 450 documents in batches. Separate checks cover the upgrade of an old SQLite database, the Word export of English and Urdu emails, the parsing of the AI's JSON answer and the reading of the Firebase key.

Table 10. *Summary of automated tests*

Test run	Checks	Result
Browser: landing page and dashboard	44	All passed
Backend with SQLite	32	All passed
Backend with the Firestore store	34	All passed
Total	**110**	**All passed**

6.4 Manual checklist for the live deployment

Table 11. *Manual test checklist*

No.	Step	Expected result
1	Open /api/health	Shows storage "firestore" and api_key_configured true
2	Generate one email with each tone	A complete email with greeting, body and sign-off in a matching tone
3	Paste an email and use Reply mode	A reply that answers the points of the pasted email
4	Generate in Urdu and download the Word file	Right-to-left text that reads correctly in Word
5	Use Check quality, Detect tone, Fix grammar and Suggest subjects	Each tool shows a sensible result without errors
6	Save a template, reload the page	The template is still there
7	Reload the page after generating emails	History and favourites are still present
8	Open the site in a private window	An empty history, proving that spaces are separate
9	Open the site on a phone	Tab bar at the bottom, no sideways scrolling
10	Use Speak in Chrome or Edge, then export to PDF	The purpose is filled by voice; the PDF contains only the email

7. Deployment

7.1 How the deployment works

Vercel is used to host the project. When the repository is imported, Vercel installs the packages listed in requirements.txt and recognises the FastAPI application in main.py, which becomes a serverless function. The folder public is served from the content delivery network, with vercel.json enabling clean addresses so that app.html is reached as /app. Data is kept in Firebase Firestore, which the server reaches with a service account key.

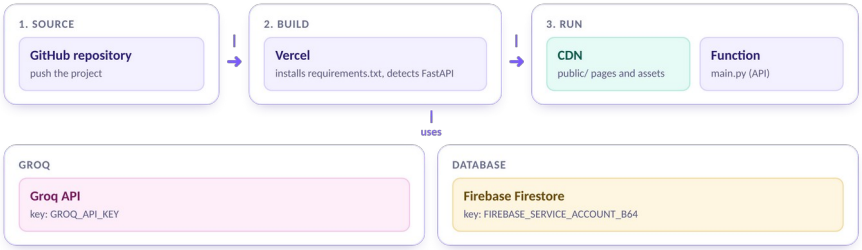


Figure 16. *Deployment: GitHub, Vercel build, CDN and function, with Groq and Firestore*

7.2 Environment variables

Table 12. *Environment variables*

Name	Required	Purpose
GROQ_API_KEY	Yes	Key for the Groq API
FIREBASE_SERVICE_ACCOUNT_B64	On Vercel	The Firebase service account key in one safe line, made with tools/encode_firebase_key.py
GROQ_MODEL	No	Overrides the model name
DATABASE_PATH	No	Overrides the location of the SQLite file

7.3 Steps

1. In the Firebase console, create a Firestore database and generate a new private key for a service account.
2. Run the encoder tool on the downloaded key and copy the line it prints.
3. Push the project to GitHub. The .gitignore file keeps .env files and Firebase keys out of the repository.
4. In Vercel, import the repository and add GROQ_API_KEY and FIREBASE_SERVICE_ACCOUNT_B64 as environment variables.
5. Deploy, then open /api/health and run the manual checklist.

7.4 Security considerations

- Secrets exist only in environment variables and in a local .env file that is never committed.
- The Firestore security rules stay locked, because only the server connects to the database.
- Cookies are HttpOnly, SameSite=Lax and Secure on HTTPS, and the identifier is validated before use.
- A public link lets anyone use the Groq key to write emails. Usage limits can be set in the Groq console, and the key can be replaced if it is misused.

8. Challenges and Solutions

Table 13. *Main challenges and their solutions*

Challenge	Why it mattered	Solution
Vercel's disk is read-only	A SQLite file cannot be written, so history and settings would be lost	A storage layer chooses Firestore in production and keeps SQLite for local work
The AI does not always return clean JSON	A broken answer would crash the page	JSON mode, a tolerant parser, validation and a clear error message
Pasted emails may contain instructions	The model could be steered by text it was only meant to answer	Prompts state that pasted text is content, not a command
A public link would share one database	Visitors could read one another's emails	A cookie gives each browser a private space
Urdu in the page, in Word and in PDF	Right-to-left text breaks easily	Right-to-left styling, a dedicated Word layout and a print-only sheet for PDF
Firestore cannot search inside text	Search is a core feature of the history	The store filters in Python, which suits personal data sizes
The page must feel alive while the AI works	Waiting without feedback feels broken	Locked controls, progress labels and a typing animation

9. Limitations and Future Work

9.1 Limitations

- The quality of every email depends on the language model, and the wording can differ between runs. Users should always read an email before sending it.
- The free Groq tier has rate limits, and the service must be reachable for the AI features to work.
- There are no user accounts. A private space belongs to one browser, and clearing cookies loses it.
- Searching in Firestore loads the visitor's emails first, which is suitable for a few hundred emails but not for very large histories.
- The quality score and the tone percentages are the model's opinion, not a measured fact.
- Voice input works only in Chromium-based browsers and needs an internet connection.

9.2 Future work

- **Accounts.** Sign-in with Google so that history follows the user across devices.
- **Rate limiting.** Per-visitor limits that protect the shared AI key.
- **Streaming.** Show the email word by word while the model writes it.
- **More languages.** Arabic, Hindi and others, with matching right-to-left support.
- **Mail integration.** Send directly through Gmail or Outlook after confirmation.
- **Team features.** Templates shared within a group.
- **Smarter replies.** Suggest meeting times from a calendar and compare subject lines.

10. Conclusion

LetterLy fulfils the assignment: a FastAPI backend takes a recipient name, a purpose and a tone, asks an LLM to write a complete email and shows the result on an HTML page. It also goes much further. The reply mode, the editing and analysis tools, the templates, the organisation features, the exports and the analytics turn a single form into a full writing assistant, while the landing page and the dashboard give it a consistent and professional appearance.

The project shows how a language model can be used responsibly in a web application: with structured prompts, strict validation of requests and answers, safe handling of pasted text and clear messages when something goes wrong. It was built to be understood and maintained, with a separate storage layer, documentation, 110 automated checks and a clear path to deployment on Vercel with Firebase.

References

- FastAPI documentation, https://fastapi.tiangolo.com
- Groq documentation, https://console.groq.com/docs
- Pydantic documentation, https://docs.pydantic.dev
- Firebase Firestore documentation, https://firebase.google.com/docs/firestore
- Vercel documentation for FastAPI, https://vercel.com/docs/frameworks/backend/fastapi
- python-docx documentation, https://python-docx.readthedocs.io
- MDN Web Docs, Web Speech API, https://developer.mozilla.org/docs/Web/API/Web_Speech_API

Appendix A. Setup Guide

These steps run LetterLy on a local computer with SQLite. Python 3.10 or newer is required.

1. Create a virtual environment

```
python -m venv venv
venv\Scripts\activate          # Windows
source venv/bin/activate       # macOS or Linux
```

2. Install the packages

```
pip install -r requirements-dev.txt
```

3. Add the Groq key

Copy .env.example to .env and set the key:

```
GROQ_API_KEY=your_groq_api_key_here
```

4. Start the server

```
uvicorn main:app --reload
```

Open http://127.0.0.1:8000 for the landing page and http://127.0.0.1:8000/docs for the interactive API documentation.

5. Push to GitHub

```
git init
git add .
git commit -m "LetterLy: AI email generator"
git branch -M main
git remote add origin https://github.com/<username>/ai-email-generator.git
git push -u origin main
```

Appendix B. Quick User Guide

Table 14. *Quick reference*

I want to	What to do
Write a new email	Open Generate Email, enter the recipient and the purpose, pick a tone and press Generate Email.
Start from a template	Press a template chip under the purpose field, or open the Templates view.
Reply to an email	Choose Reply to an email, paste the received email and press Write reply.
Change the draft	Use the buttons under the email, or type an instruction such as "make it more polite" and press Apply.
Check the draft	Press Check quality, Detect tone or Fix grammar.
Find an old email	Open History and search, or filter by favourites and tags.
Send the email	Use Copy or Mail, or download it as Word, PDF or text.
Change my name or defaults	Open Settings, change the values and press Save settings.